

Understanding py4j

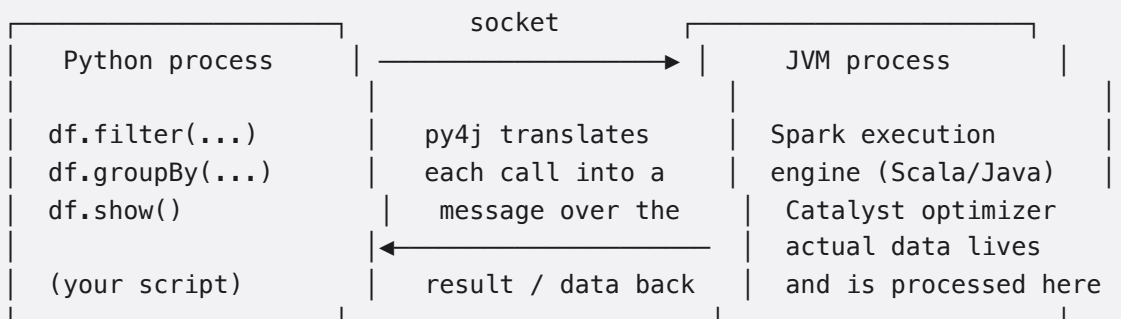
pyspark-lakehouse-aws · background reference · 2026-08-06

What it is

py4j is a generic Python↔Java bridge — a small, standalone library (not written for or by Spark) that lets a Python process control objects running inside a Java Virtual Machine. Spark itself is written in Scala and runs on the JVM; PySpark code is Python. py4j is the glue between the two.

Why Spark needs it

Python can't natively call into a running JVM process — they're two separate runtimes, two separate operating-system processes. py4j opens a local socket connection between them, and lets Python send "call this method, on this object, with these arguments" instructions across that socket.



Python never touches the real data directly — it's essentially a remote control. The DataFrame object in your Python script is a thin proxy; the actual rows, the actual query plan, the actual distributed execution all live and run on the JVM side.

What happens, step by step

1. A JVM process starts

`SparkSession.builder.getOrCreate()` starts (or connects to) a JVM process — this is the actual Spark engine.

2. The JVM opens a py4j gateway

That JVM process starts a py4j "gateway server" — a listener socket, waiting for a client to connect.

3. Python connects as a client

Your Python process connects to that gateway. From this point on, the two processes can talk.

4. Every DataFrame call becomes a py4j message

Each `.filter()`, `.groupBy()`, `.withColumn()` you write is py4j sending "call this method on this Java object" across the socket. The JVM builds up a query plan on its side. Nothing actually executes yet.

5. Materializing an action triggers real execution

Only when you call something like `.show()`, `.collect()`, or `.write` does the JVM actually run the plan — in parallel, entirely inside JVM-land — and send a result back across the socket to Python.

Where this has shown up in this project

- `$SPARK_HOME/python/lib/py4j-0.10.9.7-src.zip` — the py4j library itself, bundled inside the Spark distribution. PySpark ships its own pinned copy rather than a separately pip-installed one, because the Python-side and JVM-side py4j versions must match exactly for the socket protocol between them to work.
- The `PYTHONPATH` fix for `pytest` — `/opt/spark/python:/opt/spark/python/lib/py4j-0.10.9.7-src.zip` — added two things to Python's import path: PySpark's own package, and

py4j underneath it. `pyspark` internally does `from py4j.java_gateway import JavaGateway` the moment a `SparkSession` is created — without `py4j` importable, that fails immediately.

- `spark-submit` **"just working"** without any of this being visible — it starts the JVM, starts the `py4j` gateway inside it, and wires the connection details (port number, an auth token) to the Python driver process automatically before the script even starts running.

Example: why this makes UDFs slow

Because every Python↔JVM interaction crosses that socket, *how* you write a transformation has a real performance cost, not just a style implication.

DATAFRAME METHOD CHAIN — FAST

```
from pyspark.sql.functions import col

result = (
    orders_df
    .filter(col("status") ==
"delivered")
    .withColumn(
        "total",
        col("price") +
col("freight_value")
    )
)
result.show()
```

Each call is a lightweight `py4j` message describing an operation. The JVM assembles the full plan and runs it entirely on its own side — no data crosses the socket until `.show()` pulls back a small result.

PYTHON UDF — SLOW

```
from pyspark.sql.functions import udf
from pyspark.sql.types import
DoubleType

@udf(returnType=DoubleType())
def compute_total(price, freight):
    return price + freight

result = orders_df.withColumn(
    "total",
    compute_total(
        orders_df.price,
        orders_df.freight_value
    )
)
result.show()
```

Spark must serialize **every row** across the `py4j/socket` boundary into a Python process, run the Python function per row, then serialize the result back. That round trip, repeated per row, is the actual mechanical reason Python UDFs are slow — not just a style preference.

Key takeaway

You never write py4j code directly

PySpark hides the gateway, the sockets, and the message protocol entirely behind the DataFrame API. The only reason py4j ever surfaced directly in this project was a packaging/pathing issue — making sure Python could *locate* the bundled py4j zip file — not something requiring direct interaction. Knowing it's there explains *why* method-chaining stays fast and UDFs don't: one style stays inside the JVM, the other crosses the bridge per row.